Project Planner: one file in, a full plan out, kept up to date from site

Plan, 3 November 2026. Not built yet. Updated the same day with the decisions below.

Part A: the idea in plain words

1. You start it with one file

In a chat, you (master) attach **one file**, for example the full drawing set of the JVC school (PDF, up to 100 pages / 25 MB, or a DWG up to 35 MB), and say:

> *"Make the full plan for this project."*

2. Reem reads everything in the file

From the JVC drawings, for example, it takes:

- project name, plot, owner, consultant, approval and its date,
- floors and levels (Basement –4.10 … Roof +20.20),
- staircases, lifts, rooms and their use,
- window and door schedules (types, sizes, fire ratings),
- parking, bus bays, playground.

3. It asks only what the file doesn't say, all at once

For example: start date? budget? contractor? Do you have the structural / MEP drawings or the BOQ? You answer in one message, or attach those files too.

4. It writes the full plan and shows it on one card

Phase	What Reem writes	Quality checks before the next phase
1. Brief	Scope from the file, floors, areas, deadline, budget, who is who	Every floor and area named; dates and budget confirmed by you
2. Programme	Steps and weeks per floor (concrete, blockwork, MEP, finishes)	No step without a week; hold points on every pour and cover-up
3. Quality plan	Inspection points per stage (rebar before pour, cube tests, waterproofing, MEP pressure tests, fire doors as scheduled)	Each stage has a hold point and who signs it
4. Materials	What is needed per floor and when to order it (from the schedules in the file)	Main materials ordered at least 2 weeks before they are needed
5. Manpower	Trades and headcount per week	No week with work but no trade
6. Risks	Top risks and what to do	Each risk has an owner and an action

You tap **Start** (or say what to change). The steps go into the **JVC Project** you already have, and the full plan goes to your Shelf as a PDF.

5. Francisca and Arzhel keep it updated, from chat, as now

- "20 tons of rebar arrived", "ground floor slab poured today", "MEP stopped, no pipes".
- They confirm on the usual cards. They only report; they don't change the plan.

6. Reem watches the updates against the plan

"Late" means one of these:

1. A step hasn't started by its planned week.
2. A step isn't finished by its planned week.
3. A material isn't received in time.
4. Work went ahead without the hold-point check being signed.
5. Someone reported the work as blocked.

When that happens, Reem says what is late, by how much, what it affects, and proposes a fix ("blockwork moves to week 14 — accept?"). It buzzes **you, Francisca and Arzhel**. **Only you can accept a change to the plan.**

7. A short report every week

What's done, what's late, and what's next. It goes to you in chat and by push.

What it never does alone

It never emails, pays, or writes into saifsys, and it never changes the plan without your tap.

Money

- It uses **Sonnet 5.5 only** (no Haiku).
- Reading the file and writing the plan costs about **$3–5, once**.
- After that, Reem only thinks when someone sends an update, plus once a week for the report: a little each week.
- **Test limit: $5** for making the first plan. It stops and asks you before spending more.
- It works **7:00–22:00 Dubai time** only.
- All cost shows in Billing under "Project planner".

Who

- Starts a plan: **only the master**.
- Updates: the people already in the project (**Francisca, Arzhel**).
- Late alerts: **master + Francisca + Arzhel**.

Build in steps

Step	What you get
1	One file in chat → Reem reads it and asks its questions → plan card → Start puts the steps into the Project, PDF on Shelf
2	Late checks against the plan + push to you, Francisca and Arzhel + "accept this change?" card
3	Weekly report
4	Rules: what you correct, it remembers for the next project

First test: the JVC school, with your drawing file.

Outside services

None.

Part B: how it is built (for whoever builds it)

Goal: the master attaches one project file; Reem extracts the facts, asks for what's missing, writes a phased plan with quality checks, and, on Start, seeds the existing Project's steps. Afterwards it compares site updates against the plan and proposes changes the master accepts.

Architecture: its own module and tables (no flags on Projects). It reuses Projects for the live record: `project_steps` (weeks, hold points), `project_moves` (materials), `project_issues` / `blocked`, and the existing card flow in `project_proposals`. Site people change nothing here; they use Projects exactly as today.

Files

- `server/projectPlans.js`: tables' logic, `planKit` (master only), routes, Start → seed steps.
- `server/planRead.js`: reads the file. PDFs go through the existing reader in `knowledge.js` (100-page limit); DWG goes through `dwg.js` (35 MB). It returns the facts as JSON: project, plot, owner, consultant, approval, floors + levels, rooms, schedules (windows/doors with sizes, fire ratings), parking. Missing = list of questions.
- `server/planWatch.js`: `startPlanWatch()` timer: late checks + weekly report, under `billAs({ userId: master, job: 'project_planner' })`.
- `server/planPdf.js`: plan PDF (same helpers/fonts as `casePdf.js`).
- Client: `PlanCard.jsx` (plan + Start/Change/Cancel), `PlanChangeCard.jsx` (accept/decline a shift), a Plan tab in the project screen.
- `server/modules.js`: `{ key: 'planner', label: 'Project planner', about: 'Attach a project file; Reem writes the full plan and watches the site against it.', company: true }`.
- Model: `ADVISOR_MODEL` (Sonnet 5.5) for reading, planning and checking.
- Flag: `PLAN_WATCH=on` (pushes only off production with `LOCAL_SENDING=on`).
- Tests: `tests/project-plans.test.js`; new tables in `TABLES` in `tests/helpers/db.js`.

Tables (idempotent DDL in `server/db.js`)

```
CREATE TABLE IF NOT EXISTS project_plans (
  id SERIAL PRIMARY KEY,
  project_id INTEGER REFERENCES projects(id) ON DELETE CASCADE,
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,   -- the master who
made it
  document_id INTEGER,                       -- the one source file on the Shelf
  facts JSONB NOT NULL DEFAULT '{}',         -- what was read from the file
  answers JSONB NOT NULL DEFAULT '{}',       -- answers to the questions
  plan JSONB,                                -- phases → steps → checks, weeks, materials,
manpower, risks
  status TEXT NOT NULL DEFAULT 'reading' CHECK (status IN
    ('reading','asking','planned','started','cancelled','failed')),
  spent NUMERIC NOT NULL DEFAULT 0,
  spend_limit NUMERIC NOT NULL DEFAULT 5,
  report_doc_id INTEGER,
  started_at BIGINT,
  created_at BIGINT DEFAULT extract(epoch from now())::bigint
);
CREATE TABLE IF NOT EXISTS plan_alerts (
```

```
  id SERIAL PRIMARY KEY,
  plan_id INTEGER NOT NULL REFERENCES project_plans(id) ON DELETE CASCADE,
  step_id INTEGER REFERENCES project_steps(id) ON DELETE CASCADE,
  kind TEXT NOT NULL CHECK (kind IN
('not_started','not_finished','material','unsigned','blocked')),
  text TEXT NOT NULL,
  change JSONB,                              -- proposed shift: [{step_id, week_from,
week_to}]
  status TEXT NOT NULL DEFAULT 'open' CHECK (status IN
('open','accepted','declined','gone')),
  decided_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
  created_at BIGINT DEFAULT extract(epoch from now())::bigint,
  UNIQUE (plan_id, step_id, kind)            -- one alert per step and reason, never
repeated daily
);
```

Flow

1. **Read.** `start_plan(document_id)` (master only, one file). `planRead` extracts the facts; the facts and the questions show on one card.
2. **Ask.** Answers (or more files) fill `answers`. Status `asking` → `planned` once the planner returns a valid plan JSON (schema-checked; refused and retried once, never saved half).
3. **Check.** Before the card is shown, a separate checker call grades each phase against its quality points and fixes what fails (max 2 rounds). Total cost is held under `spend_limit`; reaching it stops and asks the master.
4. **Start.** The master taps Start → in one `tx()`, create or link the project, write `project_steps` (floor, stage, weeks, hold point, checklist points) with `steps_seeded = true`, and put the PDF on the Shelf.
5. **Watch.** Once a day at 07:00 Dubai (plus right after a confirmed site card), `planWatch` compares `project_steps`, `project_moves` and blocked/issue records against the plan, and opens at most one `plan_alert` per step and reason. Pushes go to the master + the project's members. For an alert with a `change`, the master gets an accept/decline card; accepting updates the weeks of the affected steps in one `tx()`.
6. **Report.** Sunday 08:00 Dubai: a short weekly report to the master (chat + push).

Tests

1. Only the master can start a plan or accept a change; members get 403; external users never see the module.
2. A plan JSON that fails the schema is never saved.
3. Start twice (two devices) → the steps are written once; the second tap gets 409.
4. The same late step doesn't raise a second alert the next day.
5. Accepting a change moves only the listed steps of that project.
6. Over `spend_limit` → no more AI calls for that plan until the master says go on.
7. The watch runs only within 7:00–22:00 Dubai and only with `PLAN_WATCH=on`.